

Домашнее задание № 15

1. Создаем тестовый датасет и наполняем его данными

```
-- тестовый датасет
CREATE DATABASE nyc_taxi;

CREATE TABLE nyc_taxi.trips_small (
  trip_id          UInt32,
  pickup_datetime  DateTime,
  dropoff_datetime DateTime,
  pickup_longitude Nullable(Float64),
  pickup_latitude  Nullable(Float64),
  dropoff_longitude Nullable(Float64),
  dropoff_latitude  Nullable(Float64),
  passenger_count  UInt8,
  trip_distance    Float32,
  fare_amount       Float32,
  extra             Float32,
  tip_amount        Float32,
  tolls_amount       Float32,
  total_amount      Float32,
  payment_type      Enum('CSH' = 1, 'CRE' = 2, 'NOC' = 3, 'DIS' = 4, 'UNK' = 5),
  pickup_ntaname    LowCardinality(String),
  dropoff_ntaname    LowCardinality(String)
)
ENGINE = MergeTree
PRIMARY KEY (pickup_datetime, dropoff_datetime);
```

```
-- наполняем данными
INSERT INTO nyc_taxi.trips_small
SELECT
  trip_id,
  pickup_datetime,
  dropoff_datetime,
  pickup_longitude,
  pickup_latitude,
  dropoff_longitude,
  dropoff_latitude,
  passenger_count,
  trip_distance,
  fare_amount,
  extra,
  tip_amount,
  tolls_amount,
  total_amount,
  payment_type,
  pickup_ntaname,
  dropoff_ntaname
FROM s3(
  'https://datasets-documentation.s3.eu-west-3.amazonaws.com/nyc-taxi/trips_{0..2}.gz',
  'TabSeparatedWithNames'
);
```

2. Запросы для сравнения

```
-- Запрос 1: Без использования первичного ключа
SELECT count()
FROM nyc_taxi.trips_small
WHERE passenger_count = 2;
```

Здесь фильтрация идет по passenger_count, которого нет в PRIMARY KEY (pickup_datetime, dropoff_datetime). Ожидается полное сканирование всех гранул.

⊖ --Запрос 2: С использованием первичного ключа

```
SELECT count()  
FROM nyc_taxi.trips_small  
WHERE pickup_datetime >= '2015-08-01 00:00:00'  
AND pickup_datetime < '2015-08-02 00:00:00';
```

Фильтр наложен на `pickup_datetime` — первый столбец первичного ключа. Это позволит ClickHouse использовать разреженный индекс для отсечения гранул.

3. Сравнение текстовых логов запросов

⊖ -- EXPLAIN для Запроса 1 (без PK):

```
EXPLAIN indexes = 1  
SELECT count()  
FROM nyc_taxi.trips_small  
WHERE passenger_count = 2;
```

Результат 1 ×

EXPLAIN indexes = 1 SELECT count() FROM nyc_1 | Введите SQL выражение чтобы отфильтр

	A-Z explain
4	Filter ((WHERE + Change column names to column identifiers))
5	ReadFromMergeTree (nyc_taxi.trips_small)
6	Indexes:
7	PrimaryKey
8	Condition: true
9	Parts: 3/3
10	Granules: 368/368
11	Ranges: 3

Ключевые строки, относящиеся к основному индексу:

PrimaryKey — секция индекса.

Condition: true — условие не может быть применено к индексу.

Granules: 368/368 — прочитаны все гранулы, индекс не отсек ни одной. Это означает полный скан.

-- EXPLAIN для Запроса 2 (с PK):
EXPLAIN indexes = 1
SELECT count()
FROM nyc_taxi.trips_small
WHERE pickup_datetime >= '2015-08-01 00:00:00'
AND pickup_datetime < '2015-08-02 00:00:00';

Результат 1

EXPLAIN indexes = 1 SELECT count() FROM nyc_t
Введите SQL выражение чтобы отфильтровать результаты

	A-Z explain
5	ReadFromMergeTree (nyc_taxi.trips_small)
6	Indexes:
7	PrimaryKey
8	Keys:
9	pickup_datetime
10	Condition: and((pickup_datetime in (-Inf, 1438473599]), (pickup_datetime in [1438387200, +Inf)))
11	Parts: 3/3
12	Granules: 9/368
13	Search Algorithm: binary search
14	Ranges: 4
15	ReadFromPreparedSource (_exact_count_projection)

Ключевые строки, относящиеся к основному индексу:

PrimaryKey

Keys: pickup_datetime — указано, по какому столбцу применяется индекс.

Condition: ... — видно, что условие наложено на ключевой столбец.

Granules: 9/368 — индекс отсек почти все гранулы, оставив лишь 9 из 368. Это доказывает эффективную работу основного индекса.

4. Анализ логов запросов

-- анализ логов запросов

SELECT
query,
read_rows,
read_bytes,
ProfileEvents['SelectedMarks'] AS marks_read,
ProfileEvents['SelectedMarksTotal'] AS marks_total,
round((ProfileEvents['SelectedMarksTotal'] - ProfileEvents['SelectedMarks']) / ProfileEvents['SelectedMarksTotal'] * 100, 1) AS skip_pct
FROM system.query_log
WHERE type = 'QueryFinish'
AND query LIKE '%nyc_taxi.trips_small%'
AND query NOT LIKE '%system.query_log%'
ORDER BY event_time DESC
LIMIT 2;

query_log 1

SELECT query, read_rows, read_bytes, ProfileEven
Введите SQL выражение чтобы отфильтровать результаты

	A-Z query	123 read_rows	123 read_bytes	123 marks_read	123 marks_total	123 skip_pct
1	SELECT count() FROM nyc_taxi.trips_small WHERE pickup_datetime >= '2015-08-01 00:00:00' AND pickup_datetime < '2015-08-02 00:00:00';	32 769	131 088	4	368	98,9
2	SELECT count() FROM nyc_taxi.trips_small WHERE passenger_count = 2	3 000 317	3 000 317	368	368	0

Строки логов, относящиеся к пробегу основного индекса:

marks_read (или SelectedMarks) — количество прочитанных гранул (марок).

marks_total (или SelectedMarksTotal) — общее количество гранул в таблице.

skip_pct — процент гранул, которые были пропущены благодаря индексу.

В первом запросе skip_pct = 0%, что означает полное сканирование. Во втором skip_pct ≈ 98.9%, что подтверждает эффективное использование первичного индекса для отсека данных. Эти же метрики можно увидеть в выводе EXPLAIN в строке Granules: 9/368.